# Task 1 — Single-Level Inheritance

### *Structure*

```
Person
   |
   v
Student
```

### *Requirements*

- Create a **Person** class.
- Add *name* and *age* using a constructor.
- Create a **Student** class that inherits from Person.
- Add *course* and *marks* using a constructor.
- Create a Student object.
- Display name, age, course, and marks.

### *Program*

```
# Task 1 - Single-Level Inheritance
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age


class Student(Person):
    def __init__(self, name, age, course, marks):
        super().__init__(name, age)
        self.course = course
        self.marks = marks


s = Student("Alekhya", 22, "Python Full Stack", 85)
print("Name  :", s.name)
print("Age   :", s.age)
print("Course:", s.course)
print("Marks :", s.marks)
```

### *Output*

```
Name  : Alekhya
Age   : 22
Course: Python Full Stack
Marks : 85
```

### *Explanation*

Student is the only child of Person. It calls `super().__init__(name, age)` to reuse Person's constructor and then adds its own attributes, *course* and *marks*.

**Type of inheritance:** Single-level inheritance

# Task 2 — Multi-Level Inheritance

### *Structure*

```
Animal
   |
   v
Dog
   |
   v
Puppy
```

### *Requirements*

- Create an **Animal** class with *eat()*.
- Create a **Dog** class with *bark()*.
- Create a **Puppy** class with *play()*.
- Create a Puppy object.
- Call all three methods.

### *Program*

```
# Task 2 - Multi-Level Inheritance
class Animal:
    def eat(self):
        print("Animal is eating")


class Dog(Animal):
    def bark(self):
        print("Dog is barking")


class Puppy(Dog):
    def play(self):
        print("Puppy is playing")


p = Puppy()
p.eat()
p.bark()
p.play()
```

### *Output*

```
Animal is eating
Dog is barking
Puppy is playing
```

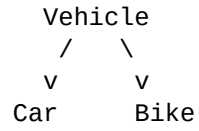
### *Explanation*

Puppy inherits from Dog, and Dog inherits from Animal, forming a chain. So a Puppy object can call *play()* (its own), *bark()* (from Dog) and *eat()* (from Animal).

**Type of inheritance:** Multi-level inheritance

# Task 3 — Hierarchical Inheritance

### *Structure*




### *Requirements*

- Create a **Vehicle** class with *start()*.
- Create a **Car** class with *drive()*.
- Create a **Bike** class with *ride()*.
- Create objects for both Car and Bike.
- Call inherited and child-specific methods.

### *Program*

```
# Task 3 - Hierarchical Inheritance
class Vehicle:
    def start(self):
        print("Vehicle is starting")


class Car(Vehicle):
    def drive(self):
        print("Car is driving")


class Bike(Vehicle):
    def ride(self):
        print("Bike is riding")


c = Car()
b = Bike()

print("--- Car ---")
c.start()    # inherited from Vehicle
c.drive()    # child-specific

print("--- Bike ---")
b.start()    # inherited from Vehicle
b.ride()     # child-specific
```

### *Output*

```
--- Car ---
Vehicle is starting
Car is driving
--- Bike ---
Vehicle is starting
Bike is riding
```

### *Explanation*

Car and Bike both inherit from the same parent, Vehicle. Both can call the inherited *start()*, while *drive()* belongs only to Car and *ride()* only to Bike.

**Type of inheritance:** Hierarchical inheritance

# Task 4 — Multiple Inheritance + MRO

### *Structure*

```
Father     Mother
    \       /
     v     v
      Child
```

### *Requirements*

- Create a **Father** class with *show()*. Print "Father".
- Create a **Mother** class with *show()*. Print "Mother".
- Create a **Child** class inheriting from both.
- Create a Child object.
- Call *show()*.
- Print the MRO using `Child.mro()`.
- Observe which *show()* method is executed.

### *Program*

```
# Task 4 - Multiple Inheritance + MRO
class Father:
    def show(self):
        print("Father")


class Mother:
    def show(self):
        print("Mother")


class Child(Father, Mother):
    pass


c = Child()
c.show()
print(Child.mro())
```

### *Output*

```
Father
[<class '__main__.Child'>, <class '__main__.Father'>, <class '__main__.Mother'>, <class
    'object'>]
```

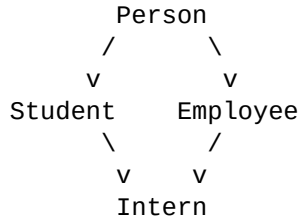
### *Explanation*

**Observation:** The output is **Father**. Child does not define *show()*, so Python uses the **Method Resolution Order (MRO)**. It checks classes left to right: Child, Father, Mother, object. Because Father is listed first in `class Child(Father, Mother)`, Father's *show()* is found first and executed. Mother's *show()* is never reached.

**Type of inheritance:** Multiple inheritance

# Task 5 — Hybrid Inheritance + MRO

### *Structure*




### *Requirements*

- Create a **Person** class with *name*.
- Create a **Student** class with *course*.
- Create an **Employee** class with *salary*.
- Create an **Intern** class inheriting from Student and Employee.
- Create an Intern object.
- Display all details.
- Print `Intern.mro()`.
- Identify the types of inheritance used.

### *Program*

```
# Task 5 - Hybrid Inheritance + MRO
class Person:
    def __init__(self, name, **kwargs):
        super().__init__(**kwargs)
        self.name = name


class Student(Person):
    def __init__(self, course, **kwargs):
        super().__init__(**kwargs)
        self.course = course


class Employee(Person):
    def __init__(self, salary, **kwargs):
        super().__init__(**kwargs)
        self.salary = salary


class Intern(Student, Employee):
    def __init__(self, name, course, salary):
        super().__init__(name=name, course=course, salary=salary)


i = Intern("Alekhya", "Python Full Stack", 15000)
print("Name  :", i.name)
print("Course:", i.course)
print("Salary:", i.salary)
print(Intern.mro())
```

### *Output*

```
Name  : Alekhya
Course: Python Full Stack
Salary: 15000
[<class '__main__.Intern'>, <class '__main__.Student'>, <class '__main__.Employee'>, <class
    '__main__.Person'>, <class 'object'>]
```

### *Explanation*

Student and Employee both inherit from Person, and Intern inherits from both Student and Employee, forming a diamond. Every class uses `super()`, so Person's constructor runs only once. The MRO is **Intern, Student, Employee, Person, object**.

**Type of inheritance:** Hierarchical (Person to Student and Employee) + Multiple (Student and Employee to Intern) = **Hybrid inheritance**